



Aula 3 — Condicionais e Loops em C

Na aula anterior, aprendemos a fazer cálculos e comparar informações.

Mas existe uma pergunta importante:

E se quisermos que o programa faça coisas diferentes dependendo do que acontecer?

Por exemplo:

- Se o aluno tiver média 10 ou mais, mostrar “Aprovado”.
- Se tiver menos de 10, mostrar “Reprovado”.
- Se a senha estiver correta, permitir o acesso.
- Se estiver errada, negar o acesso.

É aqui que entram as condicionais.

Depois vamos aprender outra coisa muito importante: como fazer o programa repetir uma tarefa várias vezes usando `for` e `while`.

1. O que são condicionais?

Uma condicional permite que o programa tome uma decisão.

Na vida real, fazemos isso o tempo todo.

Por exemplo:

“Se estiver a chover, levo um guarda-chuva.”

Em programação, fazemos algo parecido:

```
if (condicao) {  
    // código executado se a condição for verdadeira  
}
```

O `if` significa basicamente “se”.

2. Usando if

Vamos começar com um exemplo simples:

```
#include <stdio.h>  
  
int main() {
```

```

int idade;

printf("Digite a sua idade: ");
scanf("%d", &idade);

if (idade >= 18) {
    printf("Voce e maior de idade.");
}

return 0;
}

```

Se o utilizador digitar:

20

o programa mostra:

Voce e maior de idade.

Mas se digitar:

15

nada será mostrado.

Isso acontece porque a condição:

`idade >= 18`

é falsa.

3. Usando if e else

Normalmente queremos que o programa faça alguma coisa mesmo quando a condição for falsa.

Para isso usamos `else`.

```

#include <stdio.h>

int main() {

    int idade;

```



```

printf("Digite a sua idade: ");
scanf("%d", &idade);

if (idade >= 18) {
    printf("Maior de idade.");
} else {
    printf("Menor de idade.");
}

return 0;
}

```

Agora temos dois caminhos:

```

Se idade >= 18
    ↓
Maior de idade

Caso contrário
    ↓
Menor de idade

```

4. Usando else if

E se tivermos mais de duas possibilidades?

Podemos usar `else if`.

Imagine que queremos classificar uma nota:

```

10 ou mais → Aprovado
8 até 9.9  → Recuperação
menos de 8 → Reprovado

```

Podemos fazer:

```

#include <stdio.h>

int main() {

    float nota;

    printf("Digite a nota: ");
    scanf("%f", &nota);

    if (nota >= 10) {
        printf("Aprovado!");
    }
}

```

```

    } else if (nota >= 8) {
        printf("Recuperacao.");
    } else {
        printf("Reprovado.");
    }

    return 0;
}

```

Aqui o programa verifica as condições uma por uma.

Primeiro:

```
nota >= 10
```

Se for verdadeira, ele mostra "Aprovado".

Se não for, verifica:

```
nota >= 8
```

Se também não for verdadeira, chega ao `else`.

5. Exemplo real — sistema de login

Agora vamos fazer algo um pouco mais próximo de um programa real.

Podemos criar um pequeno sistema que verifica uma senha.

```

#include <stdio.h>

int main() {

    int senha;

    printf("Digite a senha: ");
    scanf("%d", &senha);

    if (senha == 1234) {
        printf("Acesso permitido.");
    } else {
        printf("Senha incorreta.");
    }

    return 0;
}

```

Esse exemplo é simples, mas mostra uma ideia importante:

O programa recebe uma informação → verifica essa informação → toma uma decisão.

É uma lógica que aparece em praticamente todo tipo de software.

6. E quando precisamos repetir alguma coisa?

Agora imagine que queremos mostrar os números de 1 até 10.

Uma forma seria escrever:

```
printf("1\n");  
printf("2\n");  
printf("3\n");  
printf("4\n");  
printf("5\n");
```

Mas imagine fazer isso até 1.000.

Seria péssimo.

É justamente para situações assim que existem os loops, também chamados de laços de repetição.

Um loop permite repetir determinado código várias vezes.

Nesta aula vamos conhecer dois:

```
for  
while
```

7. O loop for

O `for` é muito utilizado quando sabemos aproximadamente quantas vezes queremos repetir alguma coisa.

Por exemplo, mostrar os números de 1 até 10:

```
#include <stdio.h>  
  
int main() {  
  
    for (int i = 1; i <= 10; i++) {  
        printf("%d\n", i);  
    }  
}
```

```
    return 0;  
}
```

O resultado será:

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

Agora vamos entender esta parte:

```
for (int i = 1; i <= 10; i++)
```

Temos três partes.

```
int i = 1
```

É o início. Começamos com `i` valendo 1.

```
i <= 10
```

É a condição. O loop continua enquanto `i` for menor ou igual a 10.

```
i++
```

É a atualização. A cada repetição, aumentamos `i` em 1.

8. Um exemplo real com for

Imagine que queremos mostrar os produtos numerados de 1 até 5:

```
#include <stdio.h>  
  
int main() {  
  
    for (int produto = 1; produto <= 5; produto++) {
```

```

        printf("Produto %d\n", produto);
    }

    return 0;
}

```

Resultado:

```

Produto 1
Produto 2
Produto 3
Produto 4
Produto 5

```

Isso parece simples, mas a mesma lógica pode ser utilizada para percorrer listas, processar dados, gerar relatórios e muitas outras coisas.

9. O loop while

Agora vamos conhecer o `while`.

O `while` também serve para repetir código, mas funciona de uma forma um pouco diferente.

Ele significa:

“Enquanto esta condição for verdadeira, continue repetindo.”

Por exemplo:

```

#include <stdio.h>

int main() {

    int numero = 1;

    while (numero <= 5) {
        printf("%d\n", numero);

        numero++;
    }

    return 0;
}

```

Resultado:

```
1  
2  
3  
4  
5
```

Perceba que primeiro criamos a variável:

```
int numero = 1;
```

Depois temos:

```
while (numero <= 5)
```

Enquanto essa condição for verdadeira, o código dentro das chaves continua sendo executado.

E:

```
numero++;
```

faz o número aumentar.

10. for ou while?

Os dois podem fazer coisas parecidas.

Por exemplo, podemos contar de 1 até 5 usando `for`:

```
for (int i = 1; i <= 5; i++) {  
    printf("%d\n", i);  
}
```

Ou usando `while`:

```
int i = 1;  
  
while (i <= 5) {  
    printf("%d\n", i);  
    i++;  
}
```

Então, quando usar cada um?

De forma simples:

`for` costuma ser uma boa escolha quando sabemos quantas vezes queremos repetir.

`while` costuma ser útil quando queremos continuar enquanto determinada condição for verdadeira e não sabemos exatamente quantas repetições acontecerão.

11. Um exemplo real com while

Imagine um programa que só deve continuar enquanto o utilizador não digitar `0`.

```
#include <stdio.h>

int main() {

    int numero;

    printf("Digite um numero (0 para sair): ");
    scanf("%d", &numero);

    while (numero != 0) {

        printf("Voce digitou: %d\n", numero);

        printf("Digite outro numero (0 para sair): ");
        scanf("%d", &numero);
    }

    printf("Programa encerrado.");

    return 0;
}
```

Aqui não sabemos quantas vezes o utilizador vai digitar um número.

Ele pode digitar 2 números, 10 ou 100.

O programa continua funcionando até que a pessoa digite `0`.

Esse é um ótimo exemplo para entender por que o `while` pode ser útil.

12. Misturando condicionais e loops

Agora vamos juntar o que aprendemos.

Vamos criar um programa que mostra os números de 1 até 10 e informa se cada número é par ou ímpar.

```
#include <stdio.h>

int main() {

    for (int i = 1; i <= 10; i++) {

        if (i % 2 == 0) {
            printf("%d e par.\n", i);
        } else {
            printf("%d e impar.\n", i);
        }

    }

    return 0;
}
```



Resultado:

```
1 e impar.
2 e par.
3 e impar.
4 e par.
5 e impar.
6 e par.
7 e impar.
8 e par.
9 e impar.
10 e par.
```

Olha o que aconteceu aqui.

O `for` está repetindo o processo.

O `if` está tomando uma decisão em cada repetição.

E o `%` está verificando o resto da divisão.

Estamos começando a combinar várias ferramentas para criar uma lógica mais completa.

13. Pequeno projeto — Caixa eletrônico simples

Vamos terminar a aula com um pequeno projeto.

Imagine um caixa eletrônico muito simples.

O utilizador começa com 10.000 Kz e pode fazer levantamentos enquanto tiver dinheiro disponível.

```

#include <stdio.h>

int main() {

    float saldo = 10000;
    float levantamento;

    while (saldo > 0) {

        printf("\nSaldo disponivel: %.2f Kz\n", saldo);

        printf("Digite o valor que deseja levantar: ");
        scanf("%f", &levantamento);

        if (levantamento <= 0) {
            printf("Valor invalido.");
        }
        else if (levantamento > saldo) {
            printf("Saldo insuficiente.");
        }
        else {
            saldo = saldo - levantamento;
            printf("Levantamento realizado com sucesso.");
        }
    }

    printf("\nSeu saldo acabou.");

    return 0;
}

```

Aqui já temos vários conceitos trabalhando juntos:

```

while → mantém o programa funcionando
if → toma uma decisão
else if → verifica outra possibilidade
else → executa outra alternativa
- → diminui o saldo
> e <= → fazem comparações

```

Esse é o tipo de evolução que queremos ter durante o curso.

Não queremos apenas decorar comandos.

Queremos aprender a juntar os comandos para resolver problemas.

Desafio da aula

Agora tente criar um programa que peça um número ao utilizador e mostre a tabuada desse número de 1 até 10.

Por exemplo:

```
Digite um numero: 5
```

```
5 x 1 = 5
```

```
5 x 2 = 10
```

```
5 x 3 = 15
```

```
...
```

```
5 x 10 = 50
```

Tente fazer utilizando um `for`.

Depois, tente fazer a mesma coisa utilizando um `while`.

O que aprendemos nesta aula

Nesta aula aprendemos:

- O que são condicionais;
- `if`;
- `else`;
- `else if`;
- Como criar diferentes caminhos dentro de um programa;
- O que são loops;
- `for`;
- `while`;
- Como repetir código;
- Como combinar loops e condicionais;
- Como utilizar tudo isso em um pequeno projeto.

Até aqui, já conseguimos criar programas que recebem informações, fazem cálculos, verificam condições, tomam decisões e repetem tarefas.

A partir daqui, começamos a sair dos exemplos muito básicos e entrar cada vez mais na construção de pequenos projetos.

